■ Project Explainer: Patient Graphs from Cancer Data

A beginner-friendly guide — no biology background needed

■ What Is This Project, in Plain English?

Imagine you're a doctor. You have files about thousands of cancer patients — their age, what stage their cancer is at, which genes in their tumor cells are broken. The problem is: this data lives in flat spreadsheets, which are hard to reason about, hard to connect to AI, and hard to understand as a whole picture of one person.

This project turns each patient's data into a graph — like a map of connected facts — so a computer (and eventually an AI) can look at one patient holistically and reason about what's happening in their body at the molecular level.

The big-picture goal is:

> Build an AI that can explain *why* a cancer diagnosis makes sense, based on actual patient evidence, in a way that doctors can understand and trust.

This is **Milestone 1** of that plan: just building and validating the patient graphs. No AI reasoning yet.

■ Biology Glossary — Key Terms Explained Simply

Before diving in, here are all the biology terms you'll encounter, explained like you're hearing them for the first time:

Cancer & BRCA

- **Cancer**: A disease where cells in your body start growing out of control. Normal cells follow rules; cancer cells don't.
- **BRCA**: Short for *Breast Cancer*. This project focuses on breast cancer data from 1,084 patients. "BRCA" in the data actually refers to this tumor type, **not** the BRCA1/BRCA2 genes specifically.
- **TCGA**: *The Cancer Genome Atlas* — a massive US government project that collected DNA and clinical info from thousands of cancer patients and made it publicly available for research.
- **PanCancer Atlas**: A specific analysis of TCGA data that studied many cancer types together ("pan" = across all).

Genes & DNA

- **Gene**: A segment of DNA that carries instructions for making proteins. Think of it like a recipe in a recipe book. Humans have ~20,000 genes.
- **Hugo Symbol**: The official short name for a gene. For example, `TP53` is the name for a gene called "Tumor Protein p53". This naming system is maintained by a group called HUGO (Human Genome Organisation).

- **Entrez Gene ID**: A number assigned to each gene by a US database (NCBI/Entrez). Like a unique ID number for each gene. For example, `TP53` has Entrez ID `7157`.

Mutations

- **Mutation**: A change in the DNA sequence of a gene. Like a typo in a recipe. Some typos ruin the recipe; some have no effect; some accidentally make something even worse (or, rarely, better).

- **MAF (Mutation Annotation Format)**: A standardized file format that lists all detected mutations in a cancer sample. Each row in the file = one detected mutation event. This project's data has **126,252 mutation records** across all patients.

- **Variant Classification**: What *kind* of mutation it is. Examples: `Missense_Mutation` (changes one amino acid in a protein), `Nonsense_Mutation` (creates a premature stop signal — often breaks the protein), `Silent` (changes the DNA but doesn't change the protein).

- **IMPACT**: How damaging the mutation is predicted to be. Can be `HIGH`, `MODERATE`, `LOW`, or `MODIFIER`.

- **HGVSp / HGVSc**: Standard notation for where exactly the mutation is in the protein (`HGVSp`) or in the DNA code (`HGVSc`). Like a precise address for the typo.

Copy Number Alterations (CNA)

- **Chromosomes**: Humans have 23 pairs of chromosomes — long strands of DNA. Each gene sits on one of these.

- **Copy Number**: Normally, you have 2 copies of each gene (one from mum, one from dad). In cancer, cells can gain or lose copies of genes.

- **CNA (Copy Number Alteration)**: When a tumor cell has an abnormal number of copies of a gene.

- **GISTIC**: A computational algorithm that analyzes CNA data and assigns discrete scores:

- **-2**: Homozygous deletion — *both* copies of the gene are deleted. Often catastrophic.

- **-1**: Hemizygous deletion — one copy deleted, one remains.

- **0**: Neutral — normal number of copies. (We ignore these; they're not "news".)

- **+1**: Gain — extra copy, slightly more of this gene.

- **+2**: High-level amplification — many extra copies. Often drives cancer growth.

- **Why CNA matters**: If a tumor has 20 copies of an oncogene (cancer-promoting gene), it produces far too much of that protein — like cranking the volume to maximum.

Tumor Subtypes

BRCA (breast cancer) is not one disease — it's several, with very different behaviors and treatments. The **subtype** in our data: - **BRCA_Basal**: Also called "Triple Negative" — aggressive, hard to treat, no targeted therapies easily available. - **BRCA_Her2**: Overexpresses the HER2 protein — treatable with targeted drugs like Herceptin. - **BRCA_LumA** (Luminal A): Hormone receptor positive, slow-growing, good prognosis. - **BRCA_LumB** (Luminal B): Hormone receptor positive but grows faster than LumA.

Tumor Stage

The **AJCC Pathologic Tumor Stage** tells you how advanced the cancer is when diagnosed: - **Stage I**: Tumor is small, hasn't spread much. - **Stage II**: Tumor is larger or has spread to nearby lymph nodes. - **Stage III**: Significant local spread. - **Stage IV**: Cancer has spread to distant organs (metastasized). Most serious.

Aneuploidy Score

A measure of how "chromosomally chaotic" the tumor is. A high aneuploidy score means the tumor cells have lost or gained many whole chromosome segments — a common feature of aggressive cancers.

Oncotree Code / Cancer Type

- **Oncotree**: A cancer classification system. Each cancer type has a code. For example, `BRCA` = invasive breast carcinoma. This is more specific than just "breast cancer."

■ The Data Source: cBioPortal BRCA TCGA 2018

The raw data comes from **cBioPortal** — a free, open platform for exploring cancer genomics data. Specifically: the **BRCA TCGA PanCancer Atlas 2018** dataset.

This dataset contains data from **1,084 breast cancer patients**. For each patient, there are many different data files:

File	What It Contains	Size
`data_clinical_patient.txt`	Age, sex, subtype, tumor stage for each patient	1,084 rows
`data_clinical_sample.txt`	Info about each tumor sample taken	1,084 rows
`data_mutations.txt`	Every detected mutation (one row per mutation)	126,252 rows

File	What It Contains	Size
`data_cna.txt`	Copy number changes across ~25,000 genes × 1,070 samples	25,128 × 1,072 matrix
`data_mrna_seq_v2_rsem.txt`	Gene expression levels (how much each gene is "active")	20,531 genes × 1,082 samples
`data_methylation_*.txt`	DNA methylation data	22,000+ rows
`data_sv.txt`	Structural variants (big DNA rearrangements)	5,333 rows
...and more	Protein levels, phosphoprotein, RPPA, ancestry, hypoxia scores	Many files

---

■ Why We Chose the Features We Chose

With so much data available, we had to make deliberate choices about what to include. Here's the reasoning:

■ What We INCLUDED and Why

1. Clinical Patient Data

Fields: Age, Sex, Subtype, AJCC Tumor Stage, Metastasis Stage (PATH_M_STAGE), Cancer Type

Why: These are the *identity and context* of the patient. Without them, you don't know who you're looking at or how advanced their disease is. They don't predict the cancer (they describe it). They're needed for any reasoning about the patient as a person.

2. Clinical Sample Data

Fields: Sample ID, Oncotree Code, Cancer Type Detailed, Tumor Type, Aneuploidy Score, Sample Type

Why: A patient can have multiple tumor samples (e.g., primary tumor + a metastasis). Each sample has its own molecular characteristics. The Aneuploidy Score tells us how chromosomally chaotic each specific sample is.

3. Mutations (from MAF file)

Fields: Gene name, mutation type, consequence, protein change notation, genomic coordinates, impact score, filter status, number of callers that detected it

Why: Mutations are the *core molecular evidence* of what's wrong in a cancer cell. They're discrete, well-understood facts ("gene X has this specific break at this position"). We kept *every separate mutation record* as a separate edge in the graph — we didn't collapse or summarize them — because two mutations in the same gene can be very different.

Key principle: We count raw MAF rows per sample (`mutation_record_count`) but **explicitly do NOT call it clinical TMB** (Tumor Mutational Burden) because the real clinical field didn't exist in this dataset. We label it honestly.

4. Copy Number Alterations (non-neutral only)

Fields: Gene symbol, Entrez ID, CNA value (-2/-1/+1/+2) and its label

Why: CNA is the second most important molecular evidence after mutations. The GISTIC discrete scores are clean, interpretable, and directly actionable. We keep only **non-neutral** calls (skip 0s) because a neutral copy number is not evidence of anything unusual — including it would bloat the graph with irrelevant data.

Note: The raw CNA matrix had duplicate gene rows (same gene listed twice). We handled this carefully: - Exact duplicates → keep one, log the removal - Contradictory duplicates (same gene, different values for same sample) → **exclude both and log it**, because picking one would be arbitrary and potentially wrong

■ What We DELIBERATELY EXCLUDED and Why

Excluded Data	Reason
Survival outcomes (OS, DFS, PFS, DSS)	These are *results*, not *evidence*. If we include "this patient survived 5 years" as a feature, any AI trained on it would cheat — it would learn from the answer. This is called **target leakage**.
mRNA expression (20,000+ values per patient)	Too large and high-dimensional for a first-pass interpretable graph. Would make graphs enormous and harder to reason about. Deferred for later.
Methylation data	A completely different biological modality. Merging it with mutations without careful thought would mix apples and oranges.
Protein/RPPA data	Same reasoning — separate modality, much smaller coverage (only ~870 patients had it).
Genetic ancestry	Needs specific governance/fairness reasoning before including. Not required for initial biological graph.
TMB, MSI scores	These are derived summary statistics, not the same as raw mutation counts. They conceptually overlap with mutation burden but are not equivalent.
Log2 CNA, arm-level CNA	These are different representations of the same CNA data. Using multiple representations simultaneously would duplicate evidence. We chose the discrete gene-level GISTIC calls as the most interpretable.
Race/Ethnicity	Same governance concern as ancestry.
Grade, PATH_N/T stage, dates, treatment	Valid metadata but outside our initial compact feature set. The chosen stage/context attributes already cover the key clinical picture.

■■ How We Built the Patient Graphs

What Is a Graph?

A **graph** (in computer science) is a collection of: - **Nodes** (also called vertices): Things — like people, places, or concepts - **Edges**: Connections between things — relationships

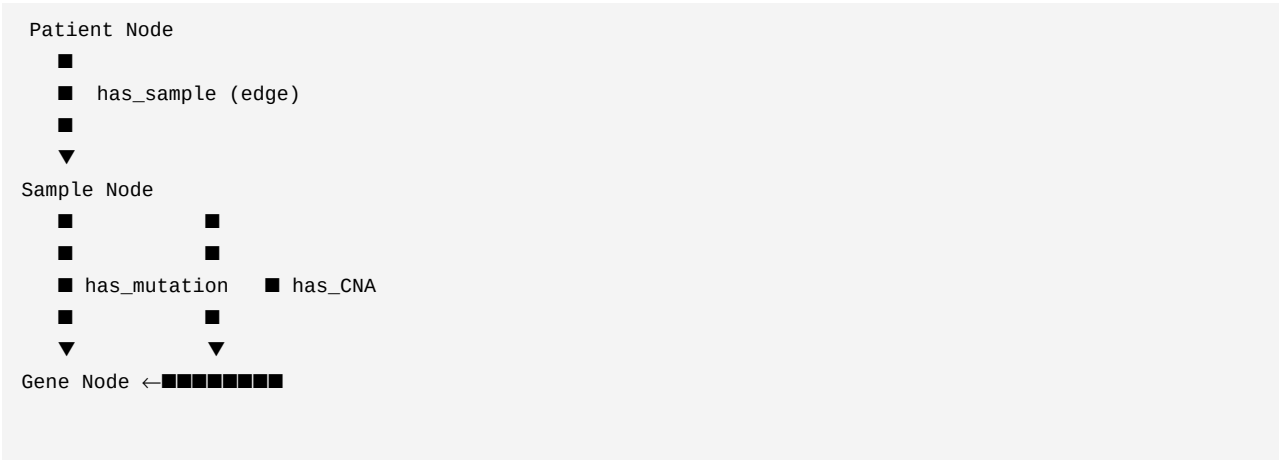
Think of a social network: people are nodes, friendships are edges.

In our case, the things are: **Patients, Tumor Samples, and Genes**. The connections are: *"this patient has this sample"*, *"this sample has this mutation in this gene"*, *"this sample has this CNA in this gene"*.

Our Specific Graph Type: `MultiDiGraph`

We use a **MultiDiGraph** from the Python library NetworkX. Breaking down the name: - **Multi**: Multiple edges can exist between the same two nodes (a sample can have many mutations in the same gene — each is a separate edge) - **Di** (Directed): Edges have a direction (arrows). Patient → Sample, not Sample → Patient - **Graph**: The structure itself

The 3-Level Structure



Every patient gets their own separate graph file. One `.graphml` file = one patient's complete molecular picture.

Step-by-Step: How a Graph Gets Built

Here's what happens when we build the graph for one patient (let's say `TCGA-AR-A0TS`):

Step 1 — Read Clinical Data

```
Load data_clinical_patient.txt  →  Find this patient's row
                                   Age, Sex, Subtype=BRCA_Basal, Stage=IIB

Load data_clinical_sample.txt  →  Find their sample(s)
                                   Sample ID: TCGA-AR-A0TS-01
```

Step 2 — Create Patient Node

```
Node: "patient:TCGA-AR-A0TS"
Attributes: {
   patient_id: "TCGA-AR-A0TS",
   subtype: "BRCA_Basal",
   ajcc_pathologic_tumor_stage: "STAGE IIB",
   age: <their age>,
   sex: "Female",
   ...
}
```

Step 3 — Create Sample Node + Connect It

```
Node: "sample:TCGA-AR-A0TS-01"
Attributes: {
   sample_id: "TCGA-AR-A0TS-01",
   aneuploidy_score: <score>,
   mutation_record_count: 100,  ← raw count of mutation rows
   ...
}

Edge: patient:TCGA-AR-A0TS → sample:TCGA-AR-A0TS-01
     (type: "has_sample")
```

Step 4 — Add Mutation Edges For each of the 100 mutations in this patient's sample:

```
 Find or create Gene node: "gene:TP53:7157"
 (gene_symbol: "TP53", entrez_gene_id: 7157)

Add Edge: sample:TCGA-AR-A0TS-01 → gene:TP53:7157
 (type: "has_mutation")
 Attributes: {
   variant_classification: "Missense_Mutation",
   impact: "HIGH",
   hgvsp_short: "p.R175H",   ← protein notation of mutation
   chromosome: "17",
   start_position: 7674220,
   ...
 }
```

Each mutation gets its **own separate edge** with its own unique ID from the original MAF file row.

Step 5 — Add CNA Edges For each non-neutral CNA call in this patient's sample:

```
 Find or create Gene node: "gene:ERBB2:2064"

Add Edge: sample:TCGA-AR-A0TS-01 → gene:ERBB2:2064
 (type: "has_CNA")
 Attributes: {
   cna_value: 2,
   cna_category: "high_level_amplification"
 }
```

Step 6 — Validate the Graph Before saving, the code checks: - ■ Does the patient node exist with the right ID? - ■ Are all clinical attributes copied correctly? - ■ Is there exactly one `has_sample` edge per sample? - ■ Do all sample→gene edges have valid types? - ■ Are there any "orphan" gene nodes (nodes with no edges going into them)? - ■ Are all CNA values valid (-2, -1, 1, or 2)? - ■ Is every mutation edge uniquely identified?

If any check fails → the build **stops and reports the error** rather than silently producing bad data.

Step 7 — Save Outputs - `TCGA-AR-A0TS.graphml` — the graph in a standard XML format any graph tool can read - `TCGA-AR-A0TS.png` — a visual drawing of the graph

---

■ The 8 Representative Patients We Built

By default, we don't build all 1,084 patients (that would create enormous files — the CNA matrix alone has 12 million non-neutral calls). Instead, we pick **8 representative patients** that span: - All 4 BRCA subtypes - High vs low mutation burden - High vs low CNA burden - A patient with zero mutations (edge case)

Here are the 8 selected patients and their graph statistics:

Patient	Subtype	Stage	Genes	Mutations	CNA Edges
TCGA-A2-A0CO	(missing)	IIB	5,496	0	5,496
TCGA-AR-A0TS	BRCA_Basal	IIB	14,314	100	14,276
TCGA-B6-A0I9	BRCA_Her2	IV	14,735	118	14,683
TCGA-BH-A0GZ	BRCA_LumA	IIA	7,758	47	7,723
TCGA-A2-A0SW	BRCA_LumB	IV	13,655	71	13,620

Patient	Subtype	Stage	Genes	Mutations	CNA Edges
TCGA-AN-A046	BRCA_LumA	IIA	10,505	7,586	6,379
TCGA-HN-A2NL	BRCA_Basal	IIA	24,430	102	24,422
TCGA-B6-A0I6	(missing)	IIA	0	0	0

Interesting observation: TCGA-AN-A046 has 7,586 mutations — far more than others. This is called a **hypermutated tumor** and is clinically significant. TCGA-HN-A2NL has 24,430 genes with CNA — extremely high chromosomal instability.

■■ What the Graphs Look Like

Here's an actual graph visualization from our output — patient **TCGA-AR-A0TS** (BRCA_Basal, Stage IIB):

And here's the most complex patient — **TCGA-HN-A2NL** (24,430 genes connected):

And the simplest — **TCGA-B6-A0I6** (no mutations or CNA data found):

■■ How the Code Is Organized

```
primekg-patient-reasoning/
■
■■■ brca_tcga_pan_can_atlas_2018/    ← Raw data files from cBioPortal
■
■■■ src/                            ← Core library code
■   ■■■ clinical.py                 ← Loads patient & sample tables; links them
■   ■■■ mutations.py                ← Loads MAF mutation data; counts per sample
■   ■■■ cna.py                      ← Loads CNA matrix; filters non-neutral; deduplicates
■   ■■■ patient_graph.py            ← Actually builds the NetworkX graph for one patient
■   ■■■ graph_schema.py             ← Defines node/edge naming conventions
■   ■■■ validation.py               ← Checks each graph is correct before saving
■   ■■■ visualization.py            ← Draws the graph as a PNG
■   ■■■ io_utils.py                 ← File reading helpers; data cleaning
■   ■■■ config.py                   ← Loads the feature policy YAML
■
■■■ scripts/
■   ■■■ build_patient_graphs.py     ← Main entry point — runs everything
■
■■■ config/
■   ■■■ features.yaml               ← Declarative list of which features to use
■
■■■ outputs/
■   ■■■ patient_graphs/*.graphml    ← The actual graph files (one per patient)
■   ■■■ visualizations/*.png        ← Graph drawings
■   ■■■ patient_summaries.csv       ← Quick stats for all built patients
■   ■■■ data_quality_report.md/json ← What was missing/malformed in the raw data
■
■■■ docs/
■   ■■■ feature_selection.md        ← Why we chose each feature
■   ■■■ data_audit.md               ← What's in every raw file
■   ■■■ graph_schema.md             ← Technical definition of the graph structure
■
■■■ tests/                          ← Automated correctness tests
```

■ The Complete Data Pipeline (End to End)

```
flowchart TD
  A["cBioPortal BRCA TCGA 2018\nRaw data files"] --> B["Data Audit\nStream every file, count rows,\nfind missing valu

  B --> C["Load Clinical Data\nclinical.py\n1084 patients, 1084 samples"]

  B --> D["Load Mutations\nmutations.py\n126,252 MAF rows"]

  B --> E["Load CNA Data\ncna.py\n25,128 genes × 1,070 samples\nFilter to non-neutral only"]

  C --> F["Link Patients to Samples\nMatch PATIENT_ID across tables"]

  F --> G["Select Representative Patients\n8 patients spanning subtypes,\nmutation burden, CNA burden"]

  D --> H["Filter to Selected Patients Only"]
  E --> H
  G --> H

  H --> I["Build Patient Graph\npatient_graph.py\nFor each patient:\n• Create Patient node\n• Create Sample node\n• A

  I --> J["Validate Graph\nvalidation.py\nCheck schema, attributes,\nno orphans, valid values"]

  J --> K{"Valid?"}

  K -->|"Yes"| L["Save Outputs\n• .graphml file\n• .png visualization"]

  K -->|"No"| M["Raise Error\nReport what failed"]

  L --> N["Write Quality Reports\n• patient_summaries.csv\n• data_quality_report.md\n• mutation_count_validation.md"]
```

■ What Comes Next (Future Milestones)

This project explicitly says "this is Milestone 1." Here's the bigger picture:

Milestone	Goal
■ Milestone 1 (done)	Build validated patient-specific graphs from BRCA clinical/mutation/CNA data
Milestone 2	Integrate **PrimeKG** — a large biomedical knowledge graph that knows about genes, diseases, drugs, and their relationships — linking our patient gene nodes to external biological knowledge
Milestone 3	Build **GNNs** (Graph Neural Networks) — AI models that can learn from the graph structure
Milestone 4	Add **neuro-symbolic reasoning** — combine statistical AI with logical rules
Milestone 5	Add **uncertainty quantification** — the AI knows what it doesn't know

Milestone	Goal
Milestone 6	Explainable diagnosis — the AI explains *why* it thinks a patient has a certain condition

Why is the gene node design important for this future work? Each gene node is named with both its Hugo symbol AND its Entrez Gene ID: `gene:TP53:7157` . PrimeKG also uses Entrez IDs to identify genes. This means in Milestone 2, we can directly match our patient graph gene nodes to PrimeKG's gene nodes — no redesign needed.

■ Key Design Decisions Explained Simply

Why MultiDiGraph instead of a simpler graph?

If sample A has 3 mutations in gene TP53, a simple graph would only allow one edge between A and TP53. A `MultiDiGraph` allows 3 separate edges — each preserving the full details of each mutation. We never want to lose information by collapsing distinct biological events.

Why keep each mutation as its own edge?

Because two mutations in the same gene can be very different. `Missense_Mutation` (changes one amino acid) vs `Nonsense_Mutation` (truncates the whole protein) in the same gene are completely different clinical situations. Collapsing them would lose crucial information.

Why not include gene expression (mRNA)?

Gene expression gives you ~20,000 numbers per patient. Including it would: 1. Make each graph enormous 2. Mix a very different type of data (continuous levels) with discrete facts (mutation yes/no, CNA category) 3. Require additional normalization choices that could introduce bias 4. Make the graph harder to interpret and debug

It's deliberately **deferred** for later, not ruled out forever.

Why validate the graph before saving?

Silent errors in data pipelines are worse than loud errors. If a sample's clinical attributes don't match what's in the graph, or if a gene node has no edges (orphan), or if mutation counts are wrong — you want to know *immediately*, not discover it months later when training a model on corrupted data.

■ Output File Formats

`.graphml`

GraphML is a standard XML-based file format for graphs. Any graph analysis tool (Gephi, NetworkX, Neo4j, etc.) can read it. Each patient's file is standalone and self-contained. The gene nodes include HGNC symbol and Entrez ID, so the files are future-proof for PrimeKG integration.

`.png` (visualizations)

A visual drawing of the graph. Because these graphs have thousands of nodes, the visualization is capped and simplified to be readable. It shows the overall structure: the hub-and-spoke pattern (one Patient → one Sample → many Genes).

`patient_summaries.csv`

A quick summary table: how many nodes, edges, mutations, CNAs each built patient has. Useful for sanity-checking and comparing patients at a glance.

`data_quality_report.md/json`

Logs of everything that was missing, malformed, or excluded from the raw data. Transparency — you always know what the data quality issues were.